

Kirutma — Distribution Guide

How to build, package, and share Kirutma with Mac and Windows users.

Version: 1.0.0 · **Stack:** Tauri 2

1. Build installable packages

Build **on each platform** (or use CI). Tauri does not reliably cross-compile between Mac and Windows from a single machine.

```
cd apps/desktop
npm install
npm run build:app
```

This runs `tauri build` with `"targets": "all"` in `apps/desktop/src-tauri/tauri.conf.json`, which produces:

Platform	What users install	Output location
Mac (Apple Silicon)	.dmg (recommended) or .app	apps/desktop/src-tauri/target/release/bundle/dmg/ and .../bundle/macos/
Mac (Intel)	Same, but build on an Intel Mac or configure a separate target	Same paths, different arch in filename
Windows	.msi and/or .exe (NSIS)	apps/desktop/src-tauri/target/release/bundle/msi/ and .../bundle/nsis/

Local Mac install (testing only)

```
cd apps/desktop
npm run install:mac
```

This builds, copies `Kirutma.app` to `/Applications/`, and opens it. For sharing with others, distribute the **DMG**, not the raw `.app` folder.

2. Build both platforms with GitHub Actions

CI already builds on `macos-latest` and `windows-latest` (see `.github/workflows/ci.yml`), but it does not publish install artifacts.

Recommended flow for releases:

1. Tag a version: `git tag v1.0.0 && git push origin v1.0.0`
2. CI builds Mac + Windows installers
3. Upload them as **GitHub Release assets**
4. Share the release URL as your download page

This is the simplest global distribution path: one link, both platforms, no store fees.

3. What to give users

Mac users

1. Download `Kirutma_1.0.0_aarch64.dmg` (Apple Silicon) or an Intel build if you ship one
2. Open the DMG
3. Drag **Kirutma** to Applications
4. Launch from Applications

Example path after a local build:

```
apps/desktop/src-tauri/target/release/bundle/dmg/Kirutma_1.0.0_aarch64.dmg
```

Windows users

1. Download the `.msi` (cleanest for IT/admin) or the NSIS `.exe` (familiar install wizard)
2. Run the installer
3. Launch from the Start menu

After building on Windows:

```
apps/desktop/src-tauri/target/release/bundle/msi/  
apps/desktop/src-tauri/target/release/bundle/nsis/
```

Optional landing page

Add a simple page with two buttons — **Download for Mac** and **Download for Windows** — linking to your GitHub Release files (or CDN).

4. Code signing (recommended before wide release)

Unsigned builds work, but users encounter security warnings.

Mac

Without signing/notarization, Gatekeeper may show:

“Kirutma cannot be opened because the developer cannot be verified”

Fix:

1. Enroll in the [Apple Developer Program](#) (\$99/year)
2. Sign the app
3. **Notarize** with Apple
4. Staple the notarization ticket to the DMG

Tauri docs: [macOS Code Signing](#)

Typical CI secrets: `APPLE_CERTIFICATE`, `APPLE_ID`, `APPLE_PASSWORD` (app-specific password), `APPLE_TEAM_ID`.

Windows

Without signing, SmartScreen may show:

“Windows protected your PC”

Fix: Obtain an **Authenticode code-signing certificate** (e.g. DigiCert, Sectigo) and sign the `.exe` / `.msi`.

Tauri docs: [Windows Code Signing](#)

For early beta with trusted testers, unsigned builds are acceptable if you explain the one-time bypass. For a public launch, signing matters significantly.

5. Mac architecture

The default Apple Silicon build produces **aarch64** artifacts (e.g. `Kirutma_1.0.0_aarch64.dmg`).

Intel Mac users need either:

- A separate **x86_64** build, or
- A **universal binary** (both architectures in one app — larger file, single download)

Many indie apps ship ARM-only first and add Intel support if demand warrants it.

6. Distribution channels

Channel	Effort	Best for
GitHub Releases	Low	Developers, early adopters, global direct download
Your website + CDN	Low–medium	Brand control, analytics
Homebrew cask (Mac)	Medium	Power users (<code>brew install --cask kirutma</code>)
winget (Windows)	Medium	Windows power users
Mac App Store / Microsoft Store	High	Mass market and built-in updates — but review rules, sandboxing, revenue share

For v1.0, **GitHub Releases plus a simple download page** is the standard approach.

7. Pre-release checklist

1. **Bump version** consistently in:
 - `apps/desktop/package.json`
 - `apps/desktop/src-tauri/tauri.conf.json`
 - `apps/desktop/src-tauri/Cargo.toml`

2. Build Mac DMG and Windows installer (locally or via CI)
 3. Test a **fresh install** on a clean Mac and a clean Windows machine (or VM)
 4. **Sign and notarize** when ready for a broad public audience
 5. Upload to GitHub Release with clear filenames, e.g.:
 - Kirutma_1.0.0_aarch64.dmg
 - Kirutma_1.0.0_x64-setup.exe
 6. Update placeholders in marketing/ ([DOWNLOAD_URL], etc.)
 7. Document install steps in README or link to this file
-

8. Quick reference

Build commands

Production build (Mac or Windows)

```
cd apps/desktop
npm run build:app
```

Local Mac install for testing

```
npm run install:mac
```

Key paths

Artifact	Path
Mac .app	apps/desktop/src-tauri/target/release/bundle/macos/Kirutma.app
Mac .dmg	apps/desktop/src-tauri/target/release/bundle/dmg/
Windows .msi	apps/desktop/src-tauri/target/release/bundle/msi/
Windows NSIS .exe	apps/desktop/src-tauri/target/release/bundle/nsis/

Related docs

- [Tauri — Distribute](#)
 - [marketing/README.md](#) — launch copy and placeholders
 - [.github/workflows/ci.yml](#) — current CI build matrix
-

9. Optional next step: automated releases

A GitHub Release workflow can build Mac and Windows installers on every version tag and attach them to a release automatically. That removes manual upload steps and keeps downloads in sync with tagged versions.